



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Università degli Studi di Padova

Laurea in Informatica

Corso di Ingegneria del Software

Anno Accademico 2025/2026



Gruppo ByteMe

byteme2025swe@gmail.com

Specifica Tecnica_G

Versione	0.0.3
Stato	In corso
Redazione	Giulia Barzon, Joseph Grant, Chiara Grossele
Verifica_G	Verificato
Approvazione	Da approvare
Proprietario	ByteMe
Uso	Esterno
Destinatari	Prof. Tullio Vardanega Prof. Riccardo Cardin

Registro dei Cambiamenti

Versione	Data	Autore	Descrizione	Verifica _G
0.0.3	15/04/2026	Chiara Grossele	Modifica struttura e aggiunte alla sezione Architettura backend	
0.0.2	14/04/2026	Joseph Grant	Aggiunta tecnologie e architettura	
0.0.1	28/03/2026	Giulia Barzon	Creazione del documento, scrittura introduzione	Tommaso Tom- bacco

Tabella 1: Registro delle versioni del documento

Indice

1	Introduzione	1
1.1	Scopo del documento	1
1.2	Approccio alla redazione	1
1.3	Scopo del prodotto	1
2	Glossario_G	1
3	Riferimenti	1
3.1	Riferimenti informativi	2
4	Tecnologie	2
4.1	Framework frontend	2
4.2	Librerie frontend	3
4.3	Linguaggio frontend	3
4.4	Tecnologie di build	3
4.5	Tecnologie di test	4
4.6	Framework api _G backend	4
4.7	Linguaggio backend	4
4.8	Server backend	4
4.9	Tecnologie deployment	5
5	Architettura backend	5
5.1	Presentation Layer	5
5.2	Application Logic	5
5.3	Business Logic	6
5.3.1	Gestione dei Prompt _G (ai_strategies.py)	6
5.3.2	Orchestrazione delle Operazioni (operation_registry.py)	6
5.3.3	Integrazione Modelli LLM (model_strategies.py)	6
5.3.4	Servizi di Supporto al Dominio (text_cleaning.py)	6
5.4	Persistence Logic	7
6	Pattern backend	7
7	Deployment	7
8	Architettura frontend	7
9	Pattern frontend	7
9.1		7

1 Introduzione

1.1 Scopo del documento

Il presente documento costituisce la **Specifica Tecnica_G** del prodotto *Second Brain*, proposto dall'azienda *Zucchetti* al gruppo ByteMe.

Il documento descrive in dettaglio:

- le **tecnologie** adottate (linguaggi, framework, librerie, strumenti);
- l'**architettura** logica e fisica del sistema, con i pattern architetturali scelti;
- i **design pattern** applicati a livello di componenti, con motivazione delle scelte;
- la **documentazione API_G** tra frontend e backend;
- i **requisiti soddisfatti** con riferimento al documento *Analisi dei Requisiti_G* v1.4.1.

Il documento è rivolto al team di sviluppo come guida implementativa e a committente e proponente come verifica_G della coerenza tra scelte progettuali e requisiti concordati.

1.2 Approccio alla redazione

Il documento viene redatto in modo **incrementale**, assicurando coerenza con gli sviluppi in corso. Non può essere considerato completo fino al rilascio della versione 1.0.0.

1.3 Scopo del prodotto

Il progetto *Second Brain* mira a sviluppare un editor di testo_G avanzato basato sul linguaggio Markdown_G e potenziato dall'integrazione di Large Language Model (LLM). L'applicazione web consentirà all'utente di:

- scrivere e visualizzare in tempo reale testi formattati in Markdown_G;
- interagire con un LLM per riassumere, correggere, riscrivere e tradurre testi;
- ottenere analisi critiche multi-prospettiche secondo il metodo dei *Sei Cappelli per Pensare* di Edward De Bono;
- delegare all'AI la stesura completa di un testo tramite Distant Writing_G.

2 Glossario_G

Per evitare ambiguità, i termini tecnici usati nel documento vengono definiti in modo più approfondito nel documento Glossario_G v2.0.0 che si può trovare nel sito web del gruppo ByteMe alla sezione Documenti Interni e raggiungibile dal seguente link:

ByteMe/Glossario_G v2.0.0

I termini che vengono considerati meritevoli di approfondimenti sono indicati con una G a pedice.

3 Riferimenti

- *Norme di Progetto_G* versione 2.0.0
ByteMe/Norme di Progetto_G v2.0.0
- Capitolato d'appalto C6: *Second Brain* (Zucchetti S.p.A.)
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>

- Regolamento del progetto didattico:
<https://www.math.unipd.it/~tullio/IS-1/2023/Dispense/PD2.pdf>

3.1 Riferimenti informativi

- *Analisi dei Requisiti_G* versione 2.0.0 (Ultimo accesso: 01/04/2026)
PB - Analisi dei Requisiti_G v2.0.0
- *Piano di Qualifica* versione 1.0.0 (Ultimo accesso: 01/04/2026)
PB - Piano di Qualifica v2.0.0
- Verbali interni ed esterni
- Glossario_G v2.0.0 (Ultimo accesso: 01/04/2026)
Documenti interni - Glossario_G v2.0.0
- Repository_G ufficiale di EasyMDE_G: <https://github.com/Ionaru/easy-markdown-editor>
(Ultimo accesso: 24/03/2026)
- Documentazione linguaggio markdown_G - CommonMark: <https://commonmark.org/>
(Ultimo accesso: 20/02/2026)
- Documentazione React_G 19: <https://react.dev>
Ultimo accesso: 27/03/2026
- Principi del Bulletproof React_G (repository_G ufficiale): <https://github.com/alan2207/bulletproof-react>
(Ultimo accesso: 27/03/2026)
- Documentazione Flask_G: <https://flask.palletsprojects.com>
(Ultimo accesso: 10/03/2026)
- Documentazione Docker_G: <https://docs.docker.com>
(Ultimo accesso: 27/02/2026)
- Documentazione TypeScript: <https://www.typescriptlang.org/docs/>
(Ultimo accesso: 27/03/2026)
- Slide del corso di Ingegneria del Software
(Ultimo accesso: 27/03/2026)
- Principi SOLID (slide prof. Cardin)
- Pattern architetturali (slide prof. Cardin)

4 Tecnologie

4.1 Framework frontend

Nome	versione	Descrizione
React _G	19.2.0	Libreria JavaScript per la costruzione di interfacce utente basate su componenti riutilizzabili.

Tabella 2: Descrizione del framework frontend

4.2 Librerie frontend

Nome	versione	Descrizione
EasyMDE _G	2.20.0	Editor Markdown _G con anteprima in tempo reale, basato su CodeMirror.
Axios	1.7.9	Client HTTP basato su Promise per effettuare richieste REST dal browser e da Node.js.
TanStack Query	-	Libreria per la gestione del fetching, caching e sincronizzazione dello stato server in React _G .
CSS Modules _G	-	Sistema di scope locale per i fogli di stile CSS, che evita conflitti tra classi in componenti diversi.
React _G Hot Toast	-	Libreria leggera per la visualizzazione di notifiche toast personalizzabili in React _G .
eslint	9.31.1	Strumento di linting per JavaScript e TypeScript, che aiuta a mantenere uno stile di codice coerente e a individuare errori.

Tabella 3: Descrizione delle librerie frontend

4.3 Linguaggio frontend

Nome	versione	Descrizione
TypeScript	-	Superset tipizzato di JavaScript che aggiunge tipizzazione statica e migliora la manutenibilità del codice.
TSX	-	Estensione di TypeScript che consente di scrivere markup JSX _G all'interno di file tipizzati, usata per i componenti React _G .

Tabella 4: Descrizione dei linguaggi frontend

4.4 Tecnologie di build

Nome	versione	Descrizione
Vite _G	7.3.1	Build tool moderno e veloce per applicazioni web, con supporto nativo a ES modules e Hot Module Replacement.

Tabella 5: Descrizione delle tecnologie di build

4.5 Tecnologie di test

Nome	versione	Descrizione
Vitest	4.1.4	Framework di testing per applicazioni Vite _G , compatibile con l'API _G di Jest e ottimizzato per TypeScript.
Pytest	7.0.0	Framework di testing per Python, con sintassi semplice e supporto a fixture, parametrizzazione e plugin.
testing library React _G	16.3.2	Libreria per il testing di componenti React _G , che fornisce utilities per interagire con i componenti in modo simile a come un utente reale lo farebbe.

Tabella 6: Descrizione delle tecnologie di test

4.6 Framework api_G backend

Nome	versione	Descrizione
Flask _G	-	Microframework Python per lo sviluppo di API _G REST, leggero ed estensibile tramite plugin.

Tabella 7: Descrizione del framework api_G backend

4.7 Linguaggio backend

Nome	versione	Descrizione
Python	3.12-slim	Linguaggio di programmazione ad alto livello, utilizzato per la logica di business e l'implementazione delle API _G .

Tabella 8: Descrizione del linguaggio backend

4.8 Server backend

Nome	versione	Descrizione
-	-	-

Tabella 9: Descrizione del server backend

4.9 Tecnologie deployment

Nome	versione	Descrizione
Docker _G	-	Piattaforma per la containerizzazione delle applicazioni, che garantisce ambienti riproducibili e portabili.

Tabella 10: Descrizione delle tecnologie di deployment

5 Architettura backend

Il sistema "Second Brain" è stato progettato adottando il pattern architetturale **layered architecture**. L'adozione di questo pattern mira a garantire la rigorosa separazione delle responsabilità e a minimizzare l'accoppiamento tra i moduli software. Il flusso di esecuzione segue un modello strettamente unidirezionale (dall'alto verso il basso): ogni strato forma un'astrazione e gestisce solo la logica pertinente.

Nel nostro sistema gli strati sono stati progettati come strati chiusi: una richiesta si sposta esclusivamente allo strato immediatamente sottostante, ignorando i dettagli implementativi dei livelli superiori o inferiori.

L'architettura logica del sistema si articola nei seguenti strati:

- **Presentation Layer:** Gestione di UI/UX, dati e richieste tramite API_G REST;
- **Application Logic:** Routing e orchestrazione del backend dell'applicazione;
- **Business Logic:** Nucleo dell'applicazione e gestione dei modelli LLM;
- **Persistence Logic:** Conservazione dei dati.

5.1 Presentation Layer

È il livello più alto dell'architettura, fisicamente separato dal backend e delegato all'applicativo Frontend. Ha il compito di gestire l'interfaccia utente e la user experience, raccoglie gli input testuali e le selezioni dell'utente e invia le richieste al livello sottostante tramite API_G REST. Questo strato non esegue alcuna elaborazione semantica o logica di generazione, delegando l'intero carico computazionale al backend.

5.2 Application Logic

Rappresenta il punto di ingresso logico dell'applicazione lato backend. Nel progetto, questo livello è interamente implementato nel modulo di routing del framework Flask_G all'interno del file app.py.

Questo strato ha diverse responsabilità:

- Agisce come REST Controller e orchestratore del flusso.
- Riceve le richieste HTTP POST provenienti dall'esterno.
- Effettua i primi controlli di validità semplici e sintattici dell'input.
- Recupera la configurazione necessaria interrogando il registro delle operazioni e traduce la richiesta esterna in chiamate alla Business Logic sottostante.

- Gestisce gli errori applicativi, catturando eventuali eccezioni di sistema per costruire e restituire una risposta formattata in un payload JSON standardizzato e sicuro per il client.

L'Application Logic si occupa esclusivamente del routing e dell'orchestrazione: non possiede alcuna conoscenza diretta delle regole di dominio, della struttura dei `promptG` o delle librerie di rete utilizzate per contattare l'Intelligenza Artificiale.

5.3 Business Logic

È il nucleo applicativo del sistema, dove risiedono le regole funzionali di Second Brain. Questo strato è rigidamente isolato dall'Application Logic: non possiede alcuna conoscenza del framework web (HTTP, Flask_G) né di come i dati verranno presentati o salvati, garantendo così il massimo disaccoppiamento. Al suo interno fa uso di pattern comportamentali (Strategy) per gestire la variabilità degli algoritmi in modo pulito ed estensibile.

I componenti principali di questo livello si dividono in servizi di dominio e servizi di integrazione:

- Gestione dei `PromptG`;
- Orchestrazione delle Operazioni;
- Integrazione Modelli LLM;
- Servizi di Supporto al Dominio.

5.3.1 Gestione dei `PromptG` (`ai_strategies.py`)

Incapsula la logica di costruzione dei contesti per l'Intelligenza Artificiale. Tramite il pattern Strategy, definisce le regole per le operazioni standard (`SimplePromptStrategy`), per le traduzioni (`TranslationStrategy`) e per l'analisi psicologica del testo con il metodo dei Sei Cappelli di De Bono (`DeBonoHatStrategy`).

5.3.2 Orchestrazione delle Operazioni (`operation_registry.py`)

Implementa il pattern Registry. Contiene le strutture dati (`OperationConfig`) che mappano dinamicamente gli identificatori delle richieste alle rispettive classi di `promptG` e di modello. Fornisce all'Application Layer gli oggetti già configurati e pronti all'uso.

5.3.3 Integrazione Modelli LLM (`model_strategies.py`)

Astrae e confina le complessità tecniche di rete per la comunicazione con i provider esterni (infrastruttura Zucchetti, LLM compatibili con standard OpenAI). Si occupa di iniettare le variabili d'ambiente (`APIG` keys e base URL configurati in `.env` e nel `dockerG-compose.yml`) e di lanciare la generazione effettiva. Grazie all'interfaccia `AIModelStrategy`, espone i modelli Llama 3.2_G e Gemma 3_G in modo polimorfo, consentendo al sistema di richiedere la generazione del testo senza doversi preoccupare dei formati di richiesta proprietari delle `APIG`.

5.3.4 Servizi di Supporto al Dominio (`text_cleaning.py`)

Un sottomodulo funzionale contenente un algoritmo basato su espressioni regolari (regex) atto a post-processare e "pulire" l'output fornito dai modelli linguistici. Rimuove preamboli discorsivi, garantendo formattazioni uniformi e rigorose prima che il dato risalga all'Application Layer.

5.4 Persistence Logic

Rappresenta lo strato inferiore dell'architettura, deputato alla conservazione e al recupero dei dati, dal quale dipendono indirettamente tutti i livelli superiori. In stretta coerenza con l'approccio orientato ai container (Docker_G) descritto nel `dockerG-compose.yml`, il backend è stato deliberatamente progettato come sistema stateless.

Non è presente alcun database_G server-side all'interno del backend. La persistenza dei documenti utente e dello storico delle generazioni IA è demandata interamente allo storage locale del client, sfruttando le Web Storage API_G.

Questa scelta assicura un drastico abbattimento dei costi infrastrutturali, una maggiore scalabilità e leggerezza dei container Docker_G e, soprattutto, la garanzia totale della privacy dei dati utente, poiché i testi elaborati non vengono mai memorizzati sui server dell'applicazione.

6 Pattern backend

7 Deployment

Per il deployment abbiamo scelto di usare Docker_G, che ci permette di creare container isolati per il frontend e il backend, garantendo coerenza tra gli ambienti di sviluppo e produzione. Ogni componente ha un proprio Dockerfile che definisce le dipendenze e la configurazione necessaria per l'esecuzione.

8 Architettura frontend

9 Pattern frontend

Per il frontend, abbiamo adottato un'architettura MVVM (Model-View-ViewModel) che si adatta bene al paradigma dei componenti di React_G.

9.1